

CODE FOR TOMORROW

Term 2 Grand Challenge 2026

Python: Language & Application | Weeks 9-11 | 10 - 24 October 2026

Submission Deadline: Friday 23 October 2026, 23:59

1. Overview

Congratulations - you've levelled up. Over the past eight weeks, you've gone beyond the basics. You now know how to structure code with functions, read and write files, think recursively, search and sort efficiently, and build entire systems with Object-Oriented Programming. You understand data structures - linked lists, binary trees, stacks, and queues. That's a serious toolkit.

The Grand Challenge runs over three Saturdays: brainstorming, building, and presenting. The best project wins a prize. There's also a Peer Choice Award - voted by your fellow students - so pay close attention during every presentation.

Group size: Work in groups of 2-4 students. We recommend 3-4 for a more balanced workload.

2. Theme: Code for Tomorrow

This term's theme is Code for Tomorrow. Singapore is a city that runs on data - from transport systems to healthcare, from education to environmental monitoring. Your challenge is to build a Python application that works with data to solve a real problem, big or small.

What makes this term different from Term 1? You now have the skills to build real, structured applications - not just scripts. Use OOP to design clean systems. Use File IO to read and persist data. Use data structures to organise information efficiently. Use algorithms to process data fast. Build something that could actually be used by real people.

Contextual Prompts

Dr. Ngo Hung-wing (Environmental Engineering, NUS) researches how technology can help cities adapt to climate change. From flood prediction systems to smart energy grids, her work shows that data-driven tools are essential for building resilient cities. Singapore, as a low-lying island nation, faces unique challenges from rising sea levels and extreme weather. How can we use code to monitor, predict, or respond to environmental challenges in our community?

Dr. Lim Sun Sun (Communications & Technology, SUTD) studies how digital inequality affects different communities in Singapore. Even as Singapore pushes toward being a Smart Nation, not everyone has the same access to digital tools and skills. From elderly residents struggling with apps to students without access to coding education, the digital divide is real. How can we build tools that bridge this gap - making technology more accessible, more inclusive, or more helpful for those who need it most?

Open Prompts

- **Social Impact:** Think of a problem in your school, community, or Singapore. Can code + data help? This is the hardest category - but if you pull it off, it will stand out.
- **Productivity Tool:** A task you do every day - can you make it faster, easier, or more organised? Think: a homework tracker, a study scheduler, a recipe manager.
- **Games:** A text-based RPG with a combat system, a quiz engine that reads questions from a file, a leaderboard system that saves high scores. Bonus points for OOP game design.
- **Science & Math:** A data analyser that reads CSV files and produces insights, a password generator, a file encryptor, a sorting algorithm visualiser.

The Goal: Don't just write code - build a system. Aim for an MVP (Minimum Viable Product) that is polished, well-documented, uses OOP meaningfully, and is crash-resistant.

3. Project Timeline

Week 9 - Saturday 10 October: Kickoff & Brainstorming

This is your planning day. By the end of the session, every group should have a locked-in idea and a clear sense of who owns what.

What happens during the session:

- Groups formed (2-4 students per group)
- Facilitated brainstorm: pitch your idea, get feedback from tutors and peers, refine it
- Ask the judges/experts questions (remember, some of the knowledge you learn from them will benefit you beyond the class!)
- Create the GitHub repository where you'll host your code
- Draft your README title and feature list before the session ends

Before next session: Sketch your class structure. Decide which classes you need, what attributes they have, and what methods they perform. Plan your file structure. No coding blind spots.

Week 10 - Saturday 17 October: Build Phase & Tutor Review

Come with working code. Even a skeleton is fine - but blank files are not. You have one week left after today.

What happens during the session:

- Tutor desk reviews: every group gets structured feedback on code quality, OOP design, scope, and logic from the tutors
- Coding time with tutors available throughout for debugging and advice
- Test your project - if it breaks there, fix it now

Before submission: Polish your code, complete your README, and test one final time. Submission opens after this session.

Week 11 - Saturday 24 October: Pitch Day

Your product launch. See Section 5 for the full Pitch Day format and judge details.

4. Submission Requirements

Deadline: Friday 23 October 2026, 23:59

Submit via GitHub. A complete submission includes all three of the following:

A. Your Code

- Must be written in Python
- Must include a main.py file as the entry point of your program
- Should be modular - use functions and classes where appropriate
- Must use OOP meaningfully (at least 2-3 well-designed classes)
- Should use File IO if your project involves data persistence (saving/loading)
- Must handle errors gracefully: unhandled crashes on bad user input will lose marks

B. Your README.md

Include a README.md file in your project. This is the front door of your project - make it count.

- Title & Description: What is this project? Why did you build it? (2-3 sentences)
- Features: A bulleted list of the main functionalities. E.g. 'Saves high scores to a file using File IO' or 'Uses a binary tree to store and search questions.'
- How to Run: The exact steps to run your project. What needs to be installed? What command starts the program? (python main.py)
- File Structure: A simple map of your project files and what each one does.

- Class Design: Briefly describe the main classes in your project and their responsibilities.
- Reflection: What was the hardest problem you faced, and how did you solve it? If you had two more weeks, what would you add?

C. How to Submit

- Copy your GitHub repo link
- Make sure your repo is public!
- Submit your link via the Google Form by 23 October, 23:59

Note: You learned Git this term in Week 3 (File IO folder has a GitHub Setup Guide). Use good version management and naming conventions - commit regularly, not just at the end!

5. Pitch Day - Saturday 24 October

Think of this as a product launch. You are not just showing code - you are selling an idea.

Format - 15 Minutes Per Group

- 3 min: Pitch & Problem Statement - What problem does your project solve? Why does it matter?
- 7 min: Live Demo - Show your project working. Walk through the key features. Show your code structure.
- 5 min: Q&A with Judges - Be prepared to explain your design decisions, challenges, and future plans.

Awards

Best Project - Awarded to the group with the highest combined judge score across all five rubric categories. The winning team takes home a small prize!

Peer Choice Award - Voted by your fellow students at the end of all presentations. One vote per person; you cannot vote for your own group. The group with the most votes wins. This group will also get a small prize.

Tips for a Great Demo

- Practice your demo at least twice before the day. Muscle memory matters under pressure.
- Have a backup plan if the internet fails - a recorded video or screenshots of your output.
- Show what happens when users type the wrong thing. Judges notice error handling.

- The demo is 7 minutes - the longest segment. Don't rush it. This is where you win.
- Show your class structure - a simple diagram or a walk-through of your main classes impresses judges.

6. How You'll Be Evaluated

We aren't just looking for 'code that works.' We're looking for code that is readable, structured, documented, and thoughtful. The rubric below reflects exactly what the judges will score.

Category	What We Look For	Weight	Max Score
Code Quality & Style	Readable code, meaningful names, comments, consistent formatting	20%	20
OOP Design	Meaningful class structure, encapsulation, appropriate use of inheritance	25%	25
Functionality	Does it work? Does it handle errors? Does it do what the README says?	25%	25
Creativity & Impact	Originality of idea, relevance to the theme, real-world usefulness	15%	15
Presentation	Clarity of pitch, quality of demo, ability to answer questions	15%	15

Total: 100 points

7. Pro Tips for Success

- Start with a plan, not with code. Spend the first 30 minutes of Week 9 sketching on paper - what classes does your program need? What data does each class hold? What methods does each class perform? Groups that plan well build faster and debug less.
- Design your classes before you write them. Think about what attributes and methods each class needs. A Player class might need name, health, inventory, and methods like take_damage() and heal(). An Item class might need name, value, and a describe() method.
- Use File IO for persistence. If your project should remember data between runs (high scores, user profiles, saved games), read and write to files. This shows you understand real-world application design.

- Commit to a small idea and execute it brilliantly. A polished quiz game with OOP design and file-based scoring beats a half-finished social network every time. Judges reward completion, error handling, clear thought process, and clean code far more than ambitious scope with rough edges.
- Use git properly. Commit regularly with clear messages. This shows good engineering practice and helps you track your progress. Don't push everything in one giant commit at the end.
- Test with bad input. What happens if the user types a string when you expect a number? What if a file doesn't exist? Good error handling is what separates a demo from a product.

Good luck. Build something you're proud of.